Mini Redis

A Redis-inspired in-memory key-value store built from scratch in Python to explore core systems programming concepts including networking, concurrency, persistence, recovery, background maintenance tasks, and storage engine design.

Features

Core Database Operations

- PUT
- GET
- DELETE

Supports storage and retrieval of arbitrary key-value pairs.

TCP-Based Client Server Architecture

The datastore runs as a TCP server and accepts client connections over sockets.

Clients can connect using:

```
nc localhost 6379
```

Example:

```
PUT A HELLO
OK

GET A
HELLO
```

```
DELETE A
OK
```

Command Parsing Layer

Commands pass through a dedicated parsing layer before execution.

Responsibilities:

- Command tokenization
- Validation
- Error handling
- Command object creation

This separates protocol handling from storage logic.

In-Memory Storage Engine

Data is maintained in memory using Python data structures.

Responsibilities:

- Key-value storage
- TTL tracking
- Expiration checks
- Persistence integration

Append Only File (AOF) Persistence

Every write operation is appended to a persistent log.

Example:

```
PUT A 1
PUT B 2
DELETE A
```

Benefits:

- Durable storage
- Crash recovery
- Audit trail of operations

Crash Recovery via Log Replay

When the server starts:

```
Load AOF

Replay Commands

Rebuild Database State
```

Example:

```
PUT A 1
PUT B 2
DELETE A
```

Results in:

```
B = 2
```

after replay.

Recovery occurs before the server begins accepting client connections, ensuring a consistent initial state.

Time-To-Live (TTL) Support

Keys can be assigned expiration times.

Example:

```
PUT SESSION TOKEN123 EX 60
```

The key expires automatically after 60 seconds.

Expiration timestamps are persisted and restored across restarts.

Lazy Expiration

Keys are checked for expiration during reads.

Flow:

```
GET SESSION

Check Expiry

Remove if Expired

Return Not Found
```

This prevents stale data from being returned.

Active Expiration Daemon

A dedicated background thread continuously scans for expired keys.

Flow:

```
Background Thread

Scan Expiry Map

Delete Expired Keys

Sleep

Repeat
```

Benefits:

- Frees memory automatically
- Removes stale data without requiring reads
- Mimics active expiration strategies used in real databases

Multi-Client Support

Multiple clients can connect simultaneously.

Architecture:

```
Accept Connection

Spawn Thread

Handle Client
```

Each client receives a dedicated worker thread.

Thread Safety

All client threads share the same Storage Engine instance.

Critical sections are protected using locks.

Protected operations:

- PUT
- GET
- DELETE

Benefits:

- Prevents race conditions
- Prevents data corruption
- Ensures consistency across concurrent clients

Background AOF Compaction

The AOF grows continuously over time.

Example:

```
PUT A 1
PUT A 2
PUT A 3
DELETE B
```

Current state:

```
A = 3
```

Compaction rewrites the log as:

```
PUT A 3
```

Benefits:

- Smaller AOF files
- Faster startup
- Reduced disk usage

Automatic Compaction Daemon

A dedicated daemon thread performs AOF compaction periodically.

Current implementation:

```
Every 10 Minutes

Compact AOF

Sleep

Repeat
```

This prevents unbounded AOF growth.

Architecture

```
Client

TCP Server

Command Parser

Command Object

Executor

Storage Engine

                    AOF Persistence

                    Expiry Tracking

Background Daemons

1. Active Expiration Worker
2. AOF Compaction Worker
```

Request Lifecycle

Example:

```
PUT USER ARCHITA
```

Flow:

```
Client
```

```
TCP Socket

Server

Parser

Validation

Command Object

Executor

Storage Engine

AOF Append

Response
```

Running The Project

Start server:

```
python3 main.py
```

Connect client:

```
nc localhost 6379
```

Project Structure

```
Mini_Redis/

    main.py
    redis_server.py
```

```
storage_engine.py
executor.py
command_parser.py
command.py
exceptions.py
db.aof
README.md
```

Concepts Demonstrated

- TCP Networking
- Socket Programming
- Client-Server Architecture
- Command Parsing
- Concurrent Programming
- Threading
- Thread Synchronization
- Mutex Locks
- In-Memory Datastores
- Storage Engine Design
- Log-Based Persistence
- Crash Recovery
- Background Workers
- Daemon Threads
- TTL-Based Expiration
- Log Compaction

Design Decisions

Why AOF Instead of JSON Snapshots?

- Records operations rather than full state
- Simplifies recovery
- Mimics write-ahead logging concepts used in databases
- Preserves command history until compaction

Why Use Locks?

Multiple client threads share the same Storage Engine.

Locks ensure:

- Safe concurrent access
- Consistent updates
- No race conditions

Why Active Expiration?

Lazy expiration only removes keys when accessed.

Active expiration:

- Frees memory proactively
- Removes stale data automatically
- Prevents buildup of expired entries

Why AOF Compaction?

Without compaction:

```
PUT A 1
PUT A 2
PUT A 3
PUT A 4
```

would permanently remain in the log.

Compaction keeps only the current state:

```
PUT A 4
```

Resulting in:

- Faster startup
- Smaller storage footprint

Why No LRU Eviction?

This project treats the AOF as the source of truth and behaves as a persistent datastore.

LRU eviction is more appropriate in cache architectures where another database exists as the primary source of truth.

The project intentionally prioritizes durability over eviction-based memory management.